PipedPeer: Zero-Configuration Distributed Computing via Recursive, Decentralised Coordination

submitted in partial fulfillment of the requirement
for the award of the Degree of

Bachelor of Technology

in

Computer Engineering

by

Yateen Vaviya (2023300251)

Vinayak Yadav (2023300264)

Sahil Gupta (2023300072)

under the guidance of

Prof. Rupali Sawant

Department of Computer Engineering

Bharatiya Vidya Bhavan's

Sardar Patel Institute of Technology

(Autonomous Institute Affiliated to University of Mumbai)

Munshi Nagar, Andheri-West, Mumbai-400058

University of Mumbai

2024–2025

Approval Certificate

This is to certify that the project entitled **PipedPeer: Zero-Configuration Distributed Computing via Recursive, Decentralised Coordination** by Yateen Vaviya, Vinayak Yadav and Sahil Gupta is approved for the partial fulfillment of the Major Project for the Final Year Project towards obtaining the Degree of Bachelor of Technology in Computer Engineering from University of Mumbai.

Project Guide	External Examiner
Name: Prof. Rupali Sawant	Name: _____
Date: _____	Date: _____
Head of Department	Principal

Seal of the Institute

Declaration

We declare that the work titled **PipedPeer: Zero-Configuration Distributed Computing via Recursive, Decentralised Coordination** is our own work carried out under the guidance of **Prof. Rupali Sawant** at Sardar Patel Institute of Technology. This work has not been submitted elsewhere for any degree.

Yateen Vaviya	**Vinayak Yadav**	**Sahil Gupta**
2023300251	**2023300264**	**2023300072**

ABSTRACT

This project presents **PipedPeer**, a system that runs an unmodified Python script on another machine on the network, with no cluster to configure and nothing installed on any participant beyond a single binary. Two problems normally stand between a developer and the idle machines around them: reproducing the script's environment elsewhere, and rewriting the program against a cluster API. PipedPeer removes both.

The environment is reproduced by building a **Nix closure** against a pinned package set, so the same script yields the same content-addressed store path on every machine and on any day. That store path is then used as a cluster-wide cache key: a worker that has seen an environment before pays nothing to set it up again. Measured on a three-node cluster, the first submission of a job took 12.00 seconds and later submissions of the same environment took 0.66 seconds, an eighteenfold reduction.

The program is distributed by **intercepting the parallel primitives it already uses**. A shim injected into the job's interpreter replaces `multiprocessing.Pool`, pandas `groupby` and `merge`, chunked file readers, NumPy linear algebra, `joblib` and torch data-parallel training. Interception is governed by a cost model that measures link bandwidth and ships work only when computing locally would cost more than moving it; dense matrix multiplication, for instance, is deliberately never shipped, because local BLAS beats the transport. Every remote path degrades to the local one, so using the cluster is never worse than not using it.

Placement filters candidate nodes on architecture and GPU, then ranks them on live load and hardware. Work is reserved with a three-phase lease so that concurrent submitters cannot oversubscribe a node, and an expired lease is how the system recovers from a crashed peer. Each job runs inside a `crun` OCI sandbox. In fault-injection testing, a worker killed in the middle of a distributed `Pool.map` left the run to complete with correct results.

Keywords: Distributed Computing, Peer-to-Peer, Transparent Interception, Nix, Content-Addressed Caching, OCI Sandboxing, Cost-Based Scheduling, Fault Tolerance.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Problem Statement

A developer with a laptop, a lab workstation and a spare desktop has a considerable amount of idle computing capacity and no practical way to use it. The frameworks that exist for distributed computation — Apache Spark [1], Ray [2], Dask [4] — all presuppose a cluster that somebody has already configured: a head node, matching software on every worker, a scheduler process, and a trusted network. For a team that simply wants a script to finish sooner, that setup costs more than the job is worth. The alternative, `ssh` together with `rsync`, means reproducing the environment by hand on every machine and receiving no scheduling, no isolation and no fault tolerance in return.

Two distinct problems sit underneath this, and they are usually attacked separately:

1. **The environment problem.** A script needs a particular interpreter and a particular set of libraries. Making a remote machine match the local one is the bulk of the work, and it is the reason "just `ssh` into it" does not scale past a single machine.
2. **The code problem.** Even with a matching environment, using more than one machine normally means rewriting the program against a cluster API. A script that took an afternoon to write becomes a project.

PipedPeer: Zero-Configuration Distributed Computing via Recursive, Decentralised Coordination addresses both without asking the user to do anything. The environment is reproduced exactly by shipping a Nix closure, and the program is distributed without modification by intercepting the parallel primitives it already uses.

1.2 Objectives

1. To run an unmodified Python script on the best available machine on the network, with results appearing in the user's working directory as though the script had run locally.
2. To reproduce the script's environment exactly on that machine, and to pay the transfer cost at most once per environment per cluster.
3. To choose the machine from live capacity, treating architecture and GPU availability as hard requirements and load, memory and hardware as a ranking.

4. To distribute the parallel work inside the script across the cluster with no code changes, and only when doing so is genuinely faster than running it locally.
5. To isolate each job, and to survive the loss of any node without losing the run.
6. To require nothing on a new machine beyond a single binary.

1.3 Scope

The following are within scope and implemented:

- A Go daemon that performs peer discovery, placement, leasing, sandboxed execution and result delivery, and a thin CLI client.
- Peer discovery by UDP multicast and broadcast on the local subnet, with a TCP health-probe sweep as a fallback and an optional standalone registry for discovery across subnets.
- Reproducible environments built as Nix closures against a pinned package set, transferred at most once per environment per cluster and cached by store path.
- Placement by hard filters on architecture, GPU, memory, cores and VRAM, followed by a score over live load and hardware, with a three-phase lease.
- Per-job isolation in a `crun` OCI sandbox, including GPU device and driver passthrough.
- Transparent interception of `multiprocessing.Pool`, pandas `groupby`, `merge` and chunked readers, NumPy linear algebra, `joblib` and scikit-learn, and torch data-parallel training, governed by a measured cost model.
- Result synchronisation of changed files only, with deletions carried in a manifest, and a live terminal dashboard.

The following are explicitly out of scope in the current implementation: authentication and transport encryption (Section 6.3), runtimes other than Python, discovery across the public internet without a registry, and any economic incentive mechanism.

1.4 Technologies Used

Table 1.1: Technologies used in PipedPeer

Technology	**Category**	**Purpose**
Go 1.25	Core language	Daemon, CLI and coordinator in one static binary
`go-chi/chi`	HTTP	Routing for the daemon API
`gorilla/websocket`	Streaming	Live stdout and stderr from a running job
Nix (pinned)	Environments	Content-addressed closures; the store path is the cache key
`crun`	Sandboxing	OCI container per job, including GPU device passthrough
`modernc.org/sqlite`	Storage	Node store, in pure Go so the binary stays static
`gopsutil`	Metrics	CPU, memory and process-tree measurements
Cobra, Viper	CLI	Command structure and configuration
Bubble Tea, Lip Gloss	TUI	Live dashboard of nodes and jobs
NATS (embedded)	Messaging	Optional alternate transport for lease messages
`zerolog`	Logging	Structured, levelled logging
Python	Interception	The shim, embedded in the binary and injected per job

1.5 Assumptions and Constraints

- Every participating machine runs the same binary; nothing else is installed on a worker.
- Workers run Linux, since `crun` and the Nix store are Linux-specific. The submitting side also builds for macOS as a client.
- The network is trusted. There is no authentication, authorisation or transport encryption. This is a deliberate scope decision, discussed in Section 6.3.
- Nix must be present on a worker to receive a closure; `pipedpeer setup` installs it.
- The submitting process must remain alive for the duration of a job: it holds the lease and receives the output stream.

Chapter 2

Literature Survey

This chapter surveys work across the areas PipedPeer draws on: lightweight sandboxing and environment portability at the edge, messaging and actor-style fault tolerance, failure-transparent programming models, portability in the Python ecosystem, scheduling, and collaborative task offloading. Each entry closes with the bearing it has on the design decisions taken in Chapter 4.

Paper 1: Web Application-Based WebAssembly Container Platform for Extreme Edge Computing

Authors: S. Sekigawa, C. Sasaki, A. Tagami **In:** Proc. IEEE GLOBECOM, 2023 [10]

Summary: A platform that deploys WebAssembly modules as isolated compute units on edge nodes, sandboxing and executing user-submitted binaries without a container daemon such as Docker. Evaluation on constrained hardware shows acceptable latency with strong isolation.

Bearing on this work: The paper's argument that a lightweight sandbox can replace a full container runtime is the one PipedPeer follows, but the conclusion differs. PipedPeer must run arbitrary CPython with native extensions, which Wasm cannot host, so it takes an OCI runtime (`crun`) directly rather than a bytecode sandbox. Section 4.6 measures what that choice costs: roughly 20 ms per job.

Paper 2: WebAssembly for Edge Computing: Potential and Challenges

Authors: M. N. Hoque, K. A. Harras **In:** IEEE Commun. Standards Mag., vol. 6, no. 4, 2022 [11]

Summary: A survey weighing Wasm's near-native speed and memory sandboxing against containers and VMs, identifying WASI immaturity, limited file-system and networking APIs, and absent GPU support as the blocking gaps.

Bearing on this work: These gaps are precisely why PipedPeer ships a complete native environment as a Nix closure rather than compiling user code to a portable bytecode. A Nix closure carries native dependencies and GPU libraries intact, which is a hard

requirement for the machine-learning workloads in Section 4.11.

Paper 3: A Cross-Architecture Evaluation of WebAssembly in the Cloud-Edge Continuum

Authors: S. Kakati, M. Brorsson **In:** Proc. IEEE/ACM CCGrid, 2024 [12]

Summary: Benchmarks Wasm runtimes across x86_64, ARM64 and RISC-V, reporting 70–90% of native performance and sub-10 ms startup, with I/O-bound workloads suffering disproportionately from WASI syscall translation.

Bearing on this work: PipedPeer targets heterogeneous architectures too, but resolves them at build time rather than at run time: the flake generator emits an architecture-specific closure, and architecture is a *hard filter* during placement (Section 4.2) rather than something the runtime papers over. Full native performance is retained.

Paper 4: Design of Elixir-Based Edge Server for Responsive IoT Applications

Authors: Y. Li, S. Fujita **In:** Proc. IEEE CANDARW, 2023 [13]

Summary: An IoT edge server built on Elixir and OTP, where supervisor trees restart failed processes automatically and the system stays available through individual crashes.

Bearing on this work: Supervision is the model PipedPeer's retry loop follows: a failed placement is retried on the next-best node with exponential backoff, and the work is never dropped. The difference is where recovery state lives. PipedPeer has no supervisor process; recovery falls out of lease expiry (Section 4.3), so there is nothing whose own failure needs supervising.

Paper 5: A Synergistic Elixir-EDA-MQTT Framework for Smart Transportation

Authors: Y. Zhou, X. Chen **In:** Computers, vol. 16, no. 3, 2024 [14]

Summary: Combines Elixir concurrency, event-driven architecture and MQTT publish-subscribe, showing lower overhead than REST for high-frequency telemetry.

Bearing on this work: An earlier iteration of PipedPeer used publish-subscribe messaging (NATS) as its dispatch transport for exactly the reasons this paper gives. Experience moved the system the other way: bulk transfer of closures, workspaces and results needs streaming and backpressure, so dispatch is now plain HTTP with a WebSocket for output,

and messaging is retained only as an optional transport for small lease messages. The paper documents the case for the road not taken.

Paper 6: WebAssembly with wasi-nn for Edge Machine Learning Inference

Authors: J. Bachmeier, V. Yussupov, J. Henß, H. Koziolek **In:** Proc. ECSA, 2025 [15]

Summary: Evaluates Wasm with wasi-nn for edge inference, finding good portability for CPU inference and immaturity for GPU acceleration.

Bearing on this work: Machine-learning jobs are a primary use case, which is why `torch` and `tensorflow` carry the largest entries in PipedPeer's memory-estimation table and why GPU passthrough is implemented at the OCI level. This paper's negative result on GPU support under Wasm supports the decision to ship a native CUDA-capable environment instead.

Paper 7: Case Study of WebAssembly Runtimes for AI Applications on the Edge

Authors: S. E. Khelifa, M. Bagaa, A. O. Messaoud, A. Ksentini **In:** 2024 [16]

Summary: Compares four Wasm runtimes for AI inference on edge devices, finding memory consumption to be the dominant constraint.

Bearing on this work: Memory as the binding constraint is the assumption behind PipedPeer's four-tier memory estimator and behind admission control: a node refuses a job it cannot hold, and the pool executor splits a request that would exceed 40% of available memory into sequential chunks instead of failing.

Paper 8: Programming Models for Failure-Transparent Distributed Systems

Authors: A. Bergström **In:** M.S. thesis, KTH Royal Institute of Technology, 2025 [17]

Summary: Surveys abstractions that hide distributed failure from application code — actors, reactive streams, supervision — handling crash and omission failures by retry, replication and migration without exposing failure semantics upward.

Bearing on this work: This is the closest match to PipedPeer's central invariant. The user's script contains no failure handling at all, so every remote path must degrade to a working local one. Section 4.9 sets out the five layers that enforce this, and Section 5.6 shows a worker being killed mid-computation without affecting the result.

Paper 9: Python Array API Standard

Authors: A. Meurer et al. **In:** Proc. SciPy, 2023 [18]

Summary: A standardised array API letting the same code run on NumPy, CuPy or JAX without modification, so scripts become portable across computational backends.

Bearing on this work: The goal is the same as PipedPeer's — portability without code change — but the mechanism is the mirror image. The Array API asks libraries to converge on one interface that users then adopt. PipedPeer changes nothing the user writes and instead substitutes the implementations behind the interfaces they already use. The comparison is drawn out in Section 5.9.

Paper 10: iDDS: Intelligent Distributed Dispatch and Scheduling

Authors: W. Guan et al. **In:** arXiv:2510.02930, 2025 [19]

Summary: Workflow orchestration that assigns tasks to heterogeneous resources from real-time CPU, memory, data-locality and dependency constraints, driven by a task DAG.

Bearing on this work: The scoring-based assignment is directly comparable to the placement function in Section 4.2. Two differences are worth stating: PipedPeer has no DAG, because it schedules whole scripts rather than task graphs, and it has no scheduler process at all, the scoring being performed in the submitting client. Data locality, which iDDS treats as first-class, is absent from PipedPeer's placement and is noted as future work.

Paper 11: Task Offloading in Multi-Hop Relay-Aided Multi-Access Edge Computing

Authors: Y. Deng, Z. Chen, X. Chen, Y. Fang **In:** IEEE Trans. Veh. Technol., vol. 72, no. 1, 2023 [20]

Summary: Formulates joint task assignment and relay selection to minimise completion time under transmission and energy constraints.

Bearing on this work: The offload-or-compute-locally trade-off is exactly what PipedPeer's cost model decides, and this paper's formulation of it as a comparison between transmission time and local computation time is the same shape as the inequality in Section 4.8. PipedPeer measures link bandwidth at run time rather than assuming a channel model.

Paper 12: Multi-Agent Deep Reinforcement Learning Based Dynamic Task Offloading

Authors: H. He, X. Yang, X. Mi, H. Shen, X. Liao **In:** Sensors, vol. 24, no. 16, 2024 [21]

Summary: Each device learns an offloading policy from local observations without a central coordinator, beating centralised heuristics on the energy-delay trade-off.

Bearing on this work: This establishes the ceiling PipedPeer's deterministic rule-based model gives up. The trade is deliberate: a learned policy needs training data and a training phase, which is incompatible with a tool whose value proposition is that a new machine joins by running one binary. It is a natural direction once a deployment produces enough history.

Paper 13: Joint Optimization of Multi-User Partial Offloading and Resource Allocation

Authors: D. Yong, R. Liu, X. Jia, Y. Gu **In:** Sensors, vol. 23, no. 5, 2023 [22]

Summary: Optimises the split between locally executed and offloaded portions of a task, finding partial offloading consistently better than all-or-nothing.

Bearing on this work: This paper describes what PipedPeer now does. Interception splits a single script's work across nodes at the granularity of `Pool` chunks, hash-shuffle buckets and matrix row blocks, while the remainder runs locally — partial offloading in the sense used here. The racing scheme in Section 4.9 goes further by keeping the local share speculative, so an unresponsive remote share costs nothing.

Paper 14: Energy-Efficient Collaborative Task Offloading in MEC

Authors: L. Chen et al. **In:** Ad Hoc Networks, vol. 169, 2025 [23]

Summary: Devices cooperate by relaying tasks toward edge servers, with a learned policy cutting total energy against non-cooperative baselines.

Bearing on this work: The symmetric model, where every device is both client and server, is PipedPeer's model too: one binary, and any node may submit or accept. PipedPeer also implements one-hop forwarding, where a node that cannot hold a request passes it to a healthier peer rather than refusing.

Paper 15: Blockchain Enabled Task Offloading in Digital Twin Vehicular Edge Networks

Authors: C. Li, Q. Chen, M. Chen, Z. Su, Y. Ding, D. Lan, A. Taherkordi **In:** J. Cloud Comput., vol. 12, no. 120, 2023 [24]

Summary: Smart contracts enforce service agreements between task submitters and edge workers, preventing free-riding in an untrusted network.

Bearing on this work: PipedPeer assumes a trusted network and has no authentication at all (Section 6.3), which is the single largest barrier to running it anywhere but a LAN. This paper describes the class of mechanism that would be needed to lift that restriction, and its commit-and-verify structure resembles the existing three-phase lease.

Paper 16: Congestion-Aware Distributed Task Offloading Using Graph Neural Networks

Authors: Z. Zhao, J. Perazzone, G. Verma, S. Segarra **In:** arXiv:2312.02471, 2023 [25]

Summary: Trains a graph neural network to produce distributed offloading policies from local neighbourhood observations, outperforming greedy baselines under congestion.

Bearing on this work: PipedPeer scores each node independently on scalar metrics and is therefore blind to topology: two nodes with identical load score identically even if one is a slow hop away. Inter-node bandwidth is measured, but only in the shim's cost model and not during placement. Feeding a graph-structured view into placement is the most concrete extension this survey suggests.

Chapter 3

Analysis

3.1 System Architecture

PipedPeer follows a **daemon, not CLI** architecture. Every machine runs one long-lived daemon that handles discovery, admission, execution and result delivery. The `pipedpeer` command is a short-lived client.

One consequence is worth stating early because it distinguishes the system from every framework in Chapter 2: **the coordinator is a library linked into the client, not a service**. There is no scheduler process, no head node and no master anywhere. A machine acts as submitter and worker simultaneously, and the same binary does both. A new participant joins by running that binary; nothing is registered, configured or installed.

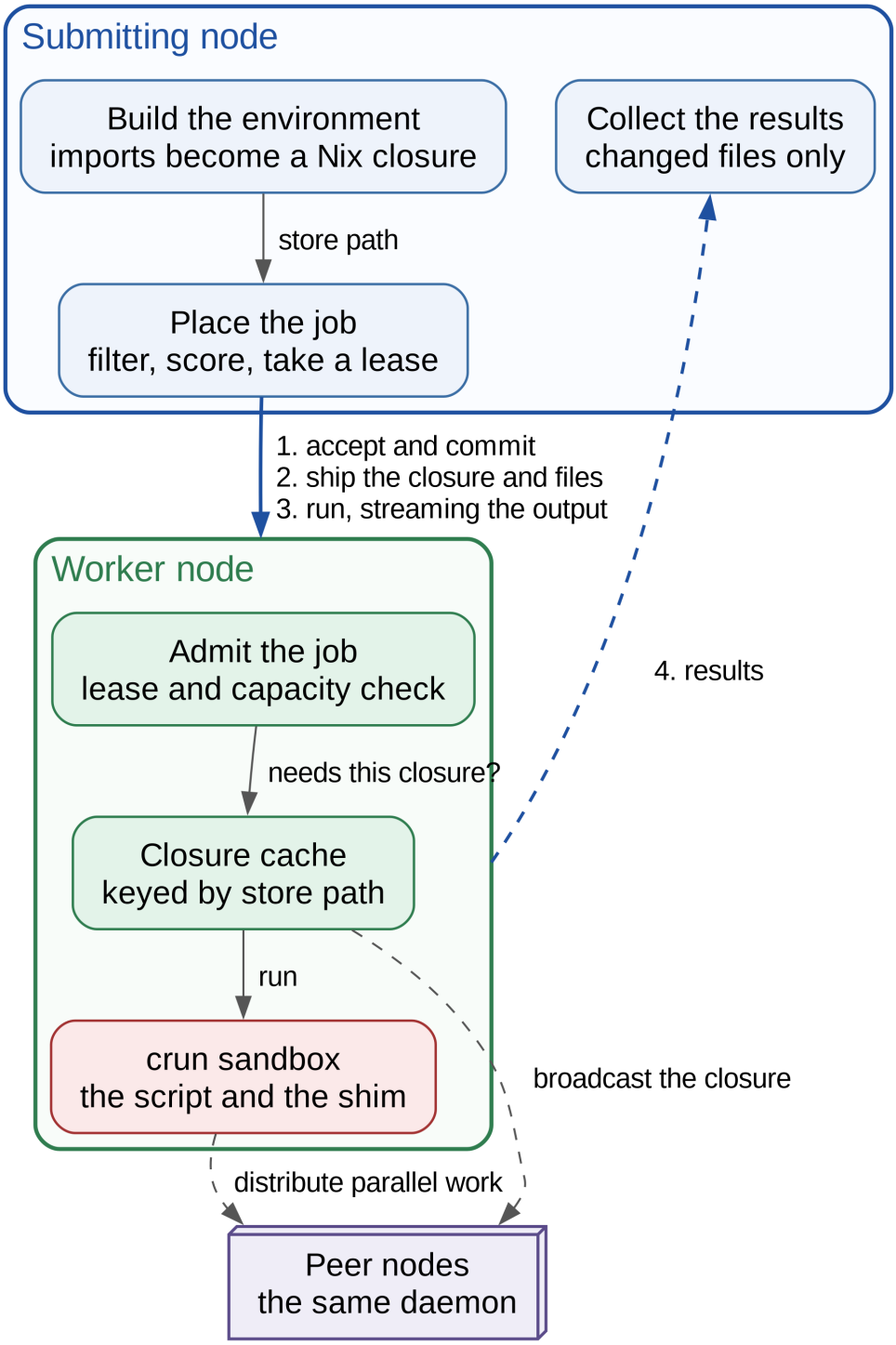

Figure 3.1: PipedPeer system design. The submitting side builds the environment and places the job; the worker admits it, caches the closure and runs it sandboxed. Both roles are the same binary.

The subsystems are listed in Table 3.1.

Table 3.1: Subsystems and their responsibilities

Package	Responsibility
`coordinator`	Filters, scores and selects a node; takes the lease; retries on failure
`daemonapi`	HTTP and WebSocket surface, lease manager, sandbox, pool executor, gradient exchange
`discovery`	UDP multicast and broadcast probes, TCP health sweep fallback
`nodestore`	SQLite record of known peers and their last observed capacity
`nixgen`	Flake generation and the interception shim as an embedded asset
`pythondeps`	Import scanning and import-to-package resolution
`resourceest`	Four-tier memory estimate
`jobhistory`	Per-job record, progress stage and result manifest
`heartbeat`	Capability and load reporting
`gpu`	Vendor detection and per-device telemetry
`registry`	Optional standalone directory for discovery across subnets

3.2 Data Flow Diagram

Level 0 — Context Diagram

At the highest level the system has three external entities: the **user**, who submits a script and receives output and result files; **peer nodes**, which supply capacity and return computed results; and an optional **registry**, which assists discovery when nodes are not on one subnet.

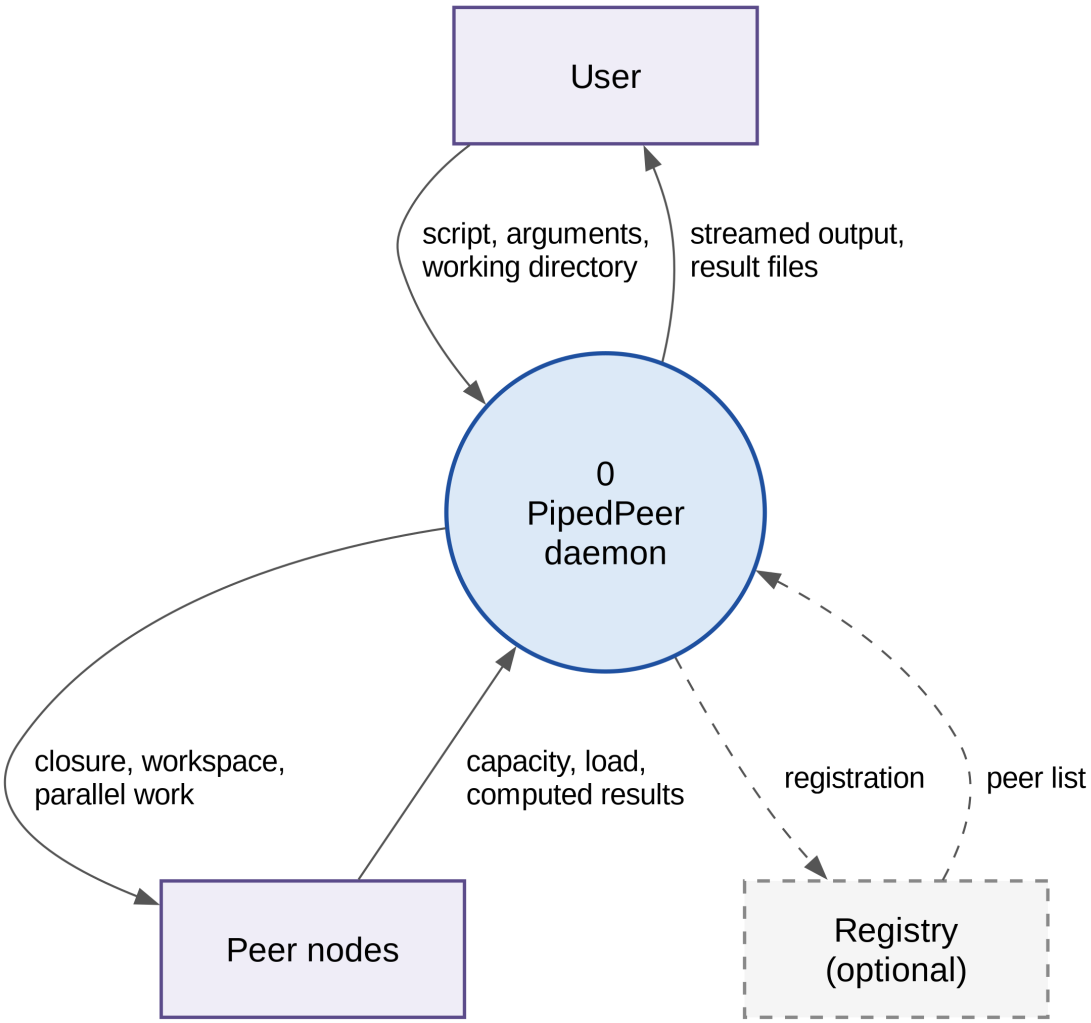

Figure 3.2: Level 0 DFD — context diagram. The registry is dashed because the system works without it.

Level 1 — System Decomposition

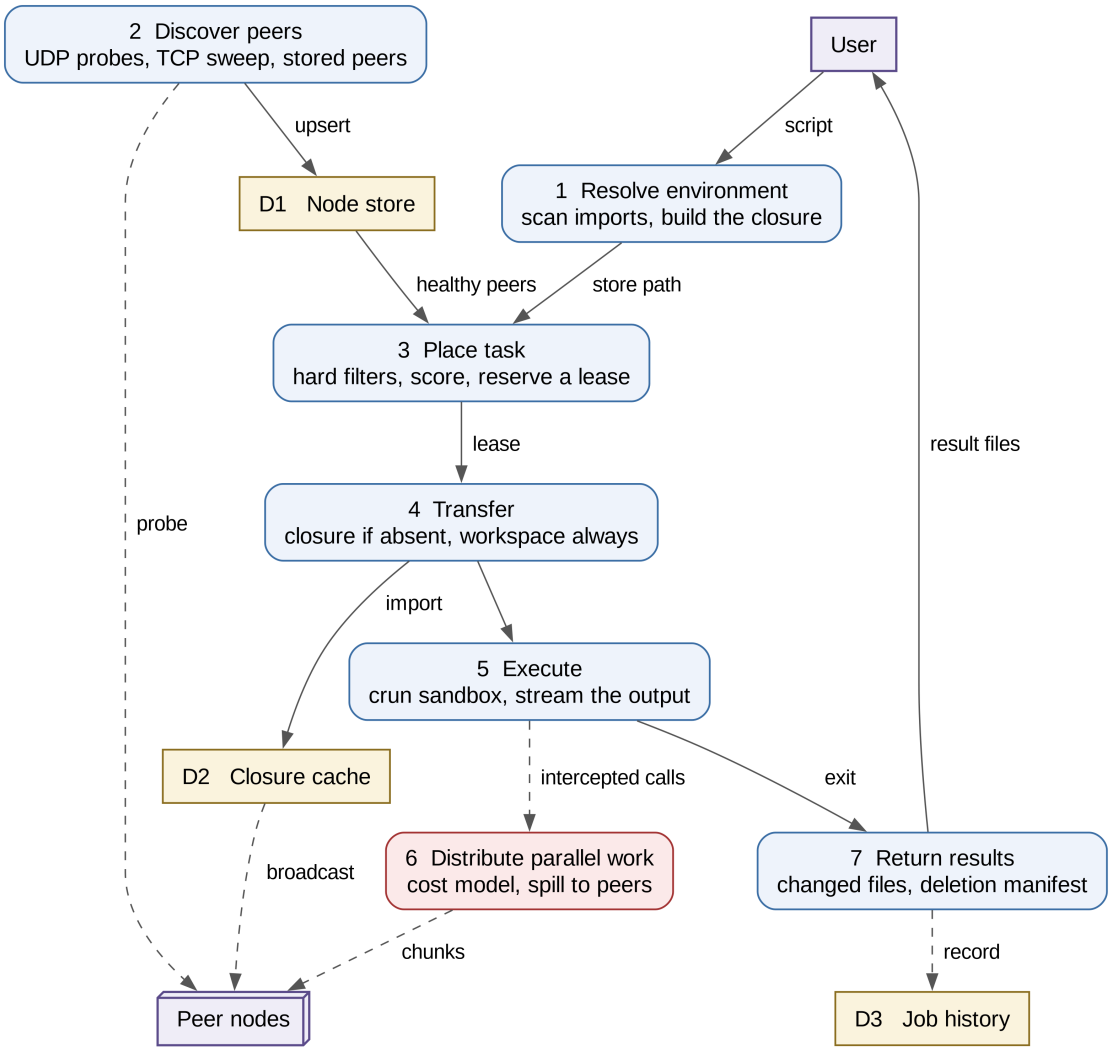

Figure 3.3: Level 1 DFD — seven processes over three data stores.

- **1 Resolve environment.** Scan the script's imports, generate a flake, build the closure.
- **2 Discover peers.** Probe the subnet and merge results into the node store.
- **3 Place task.** Apply hard filters, score the survivors, reserve a lease on the best.
- **4 Transfer.** Upload the workspace always, the closure only when the worker lacks it.
- **5 Execute.** Run inside a `crun` sandbox, streaming output back line by line.
- **6 Distribute parallel work.** Consult the cost model and spill chunks to peers.
- **7 Return results.** Send back changed files and a deletion manifest.

3.3 Use Case Diagram

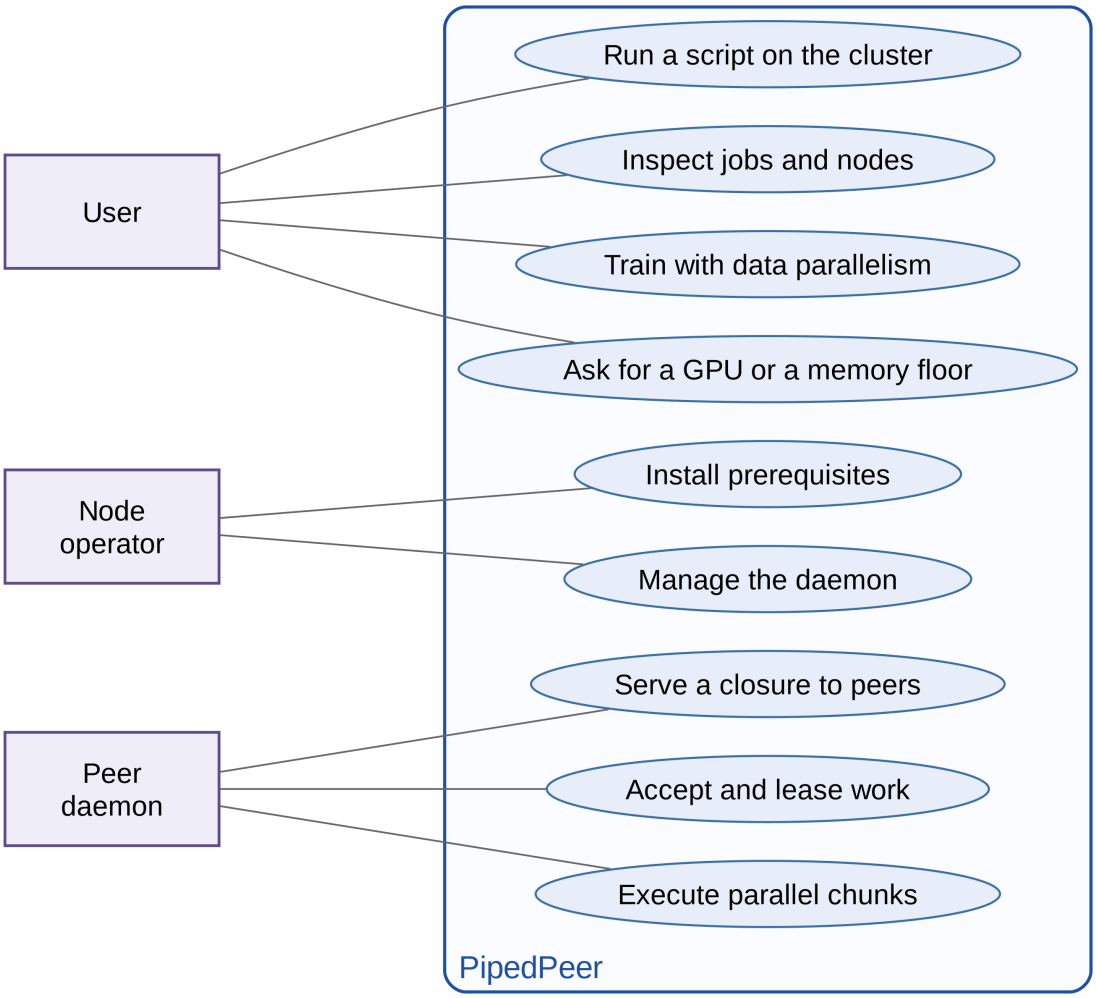

Figure 3.4: Use cases. A peer daemon is an actor in its own right, because every node serves closures and executes chunks for others.

3.4 Sequence Diagram

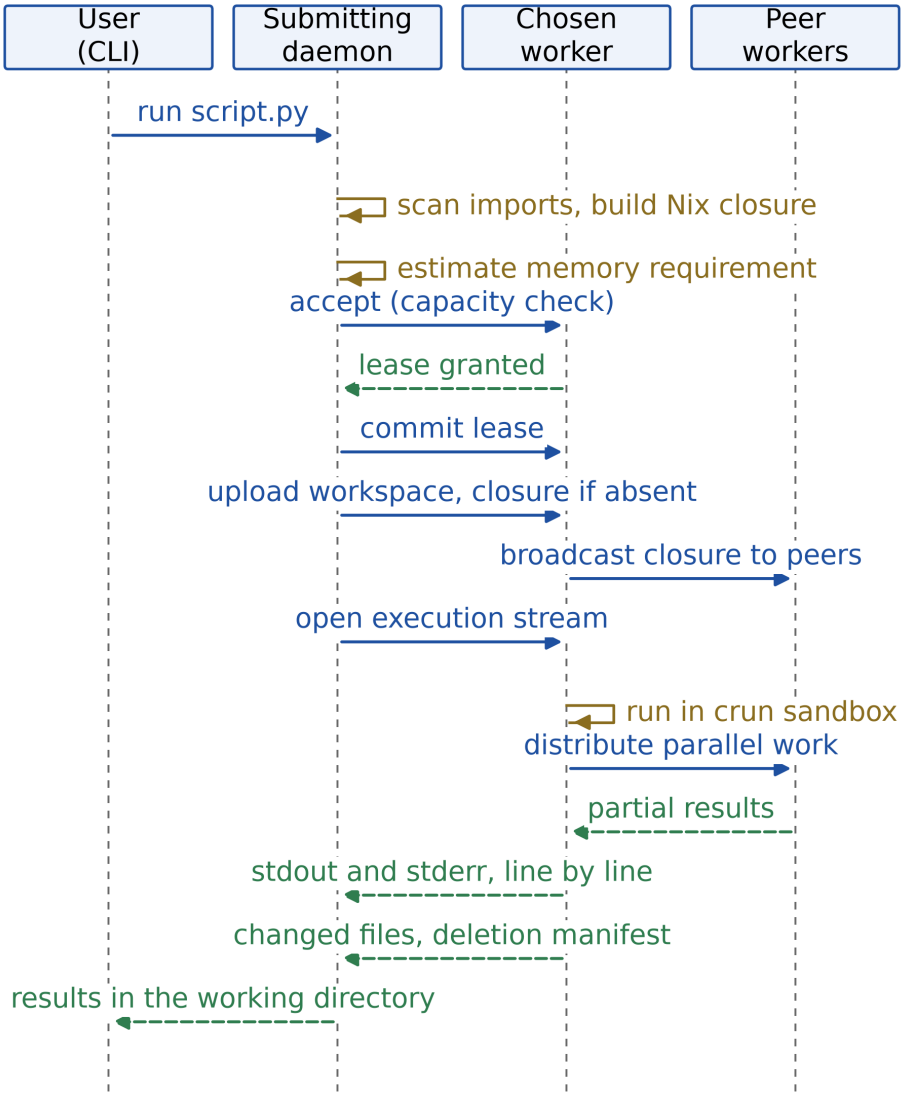

Figure 3.5: End-to-end execution of one job.

1. The user runs `pipedpeer run script.py`.
2. The client scans imports and builds a Nix closure locally, then estimates the job's memory requirement.
3. It asks its own daemon for the list of healthy peers.
4. The coordinator filters and scores them and sends `accept` to the best candidate, which checks its own capacity.
5. On acceptance the client sends `commit`, atomically reserving capacity on that node.
6. It uploads the workspace, and the closure only if the worker does not already hold it.

7. The worker broadcasts the closure to peers that lack it, so they become eligible to receive spilled work.
8. Execution proceeds inside a sandbox, with stdout and stderr streamed back a line at a time.
9. If the script uses a parallel primitive, the shim distributes it subject to the cost model.
10. On exit, changed files and a deletion manifest return as an archive and are extracted into the user's directory.

Chapter 4

Design and Methodology

4.1 Node Discovery

Three sources feed one table, and none blocks the others.

UDP probes. The daemon opens two sockets on port `38099`, one joined to multicast group `239.255.0.99` and one bound for broadcast, both with `SO_REUSEADDR` and `SO_REUSEPORT`. The probe is a fixed magic string and the reply carries a JSON node descriptor. The multicast socket exists for a specific reason: a UDP broadcast to a shared port is delivered by the kernel to only one socket, so two daemons on the same host would never see each other. Multicast with port reuse reaches all of them.

This mechanism is often described as mDNS, including in earlier documentation of this project. It is not: there is no DNS-SD, no service record and no conflict resolution. It is a bare request-response probe.

TCP fallback. Access points with client isolation drop broadcast traffic without error. When the UDP probe returns nothing, the daemon sweeps each local subnet with concurrent health probes, 64 at a time with a 700 ms timeout each.

Freshness. A peer poller runs every 10 s. Two thresholds apply, and they differ deliberately: after **45 seconds** without a successful health check — four missed rounds — a node stops being reported as healthy, which is the filter every other component reads; after **5 minutes** an auto-discovered row is deleted outright. Manually added peers are never removed by the second rule.

4.2 Placement and Scoring

Candidates are merged from the node store, live discovery, the optional registry and the local node, deduplicated by node identifier. Five hard filters then run in order: architecture must match the closure; a GPU must be present if one was required; and free memory, free cores and free VRAM must each satisfy the request. Every rejection records a reason, which surfaces in `pipedpeer job`.

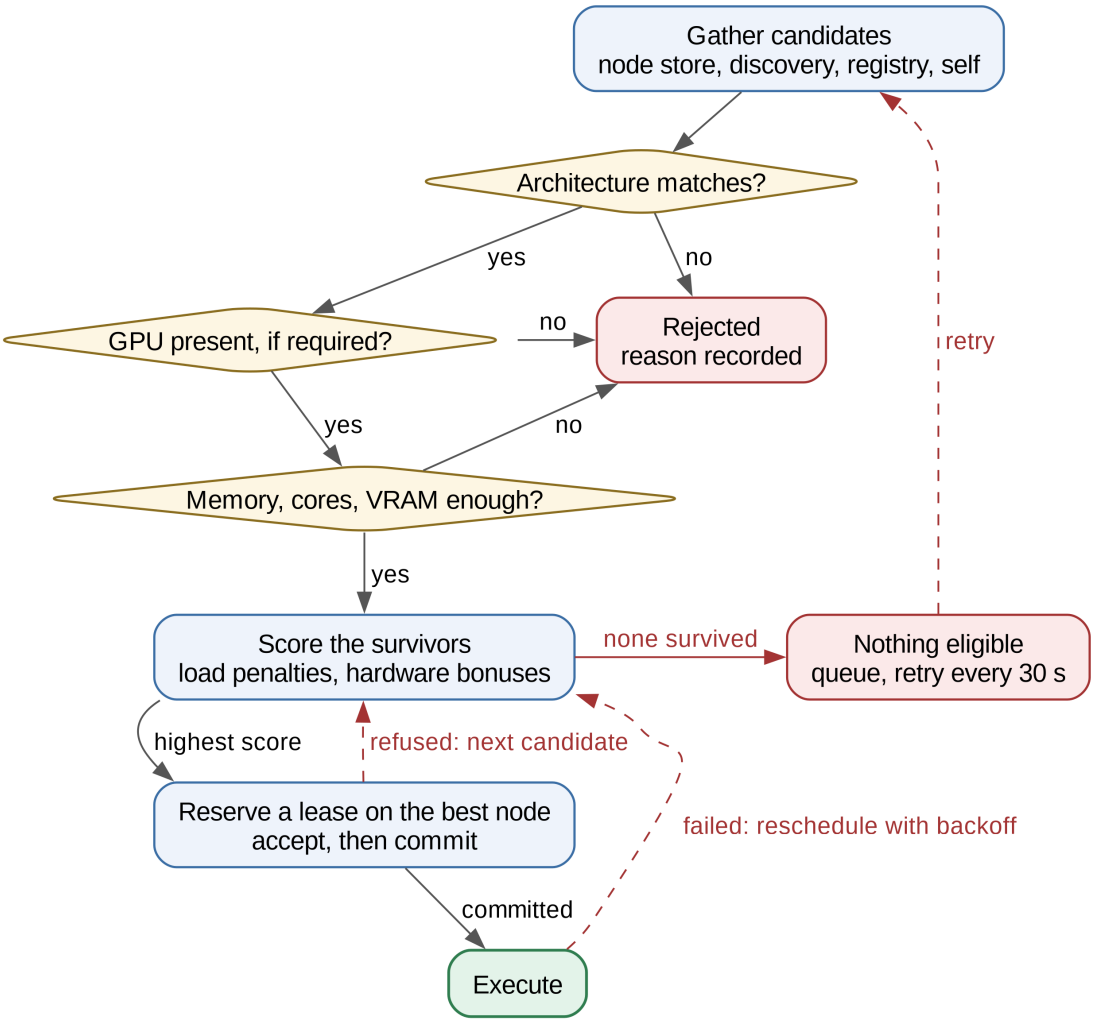

Figure 4.1: Placement: gather, filter, score, lease, retry. A task is never dropped; when nothing is eligible it is queued and retried.

Survivors are scored. Writing u_{cpu} and u_{mem} for CPU and memory utilisation in percent, n_{jobs} for the count of running jobs, and $\hat{c}, \hat{f}, \hat{m}, \hat{r}$ for the core count, clock speed, free memory and free-core ratio each normalised onto $[0, 1]$ against a saturation point:

$$S = \left(1 - \tfrac{u_{\text{cpu}}}{200} - \tfrac{u_{\text{mem}}}{200} - 0.05\, n_{\text{jobs}} + 0.10(\hat{c} - \tfrac{1}{2}) + 0.06(\hat{f} - \tfrac{1}{2}) + 0.10(\hat{m} - \tfrac{1}{2}) + 0.10(\hat{r} - \tfrac{1}{2})\right) \cdot H \tag{4.1}$$

where H is the node's health score. Two properties matter more than the individual coefficients.

Load dominates hardware. The two load terms can subtract 1.0 between them, while the four hardware terms together move the score by at most 0.36. A fast but busy machine therefore loses to a slower idle one, which is the intended behaviour on a network of workstations that people are also using.

Missing telemetry is neutral. The normalisation returns 0.5, the midpoint, for a value that is absent or non-positive, so a node that does not report its clock speed is neither rewarded nor punished. This is what makes the function safe across heterogeneous machines whose telemetry is uneven.

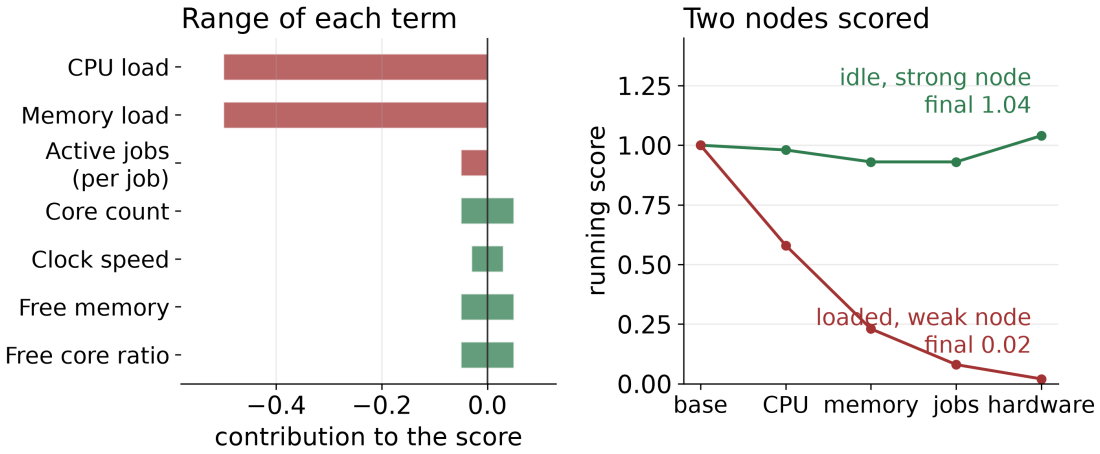

Figure 4.2: Left: the range each term can contribute. Right: the same function applied to an idle strong node and a loaded weak one.

When a GPU is requested, a second function contributes up to 0.8 more, drawn from total VRAM, device count, free VRAM on the best device and compute capability, less a utilisation penalty.

4.3 The Lease Protocol

Placement is a three-phase handshake, and its state machine is where crash recovery lives.

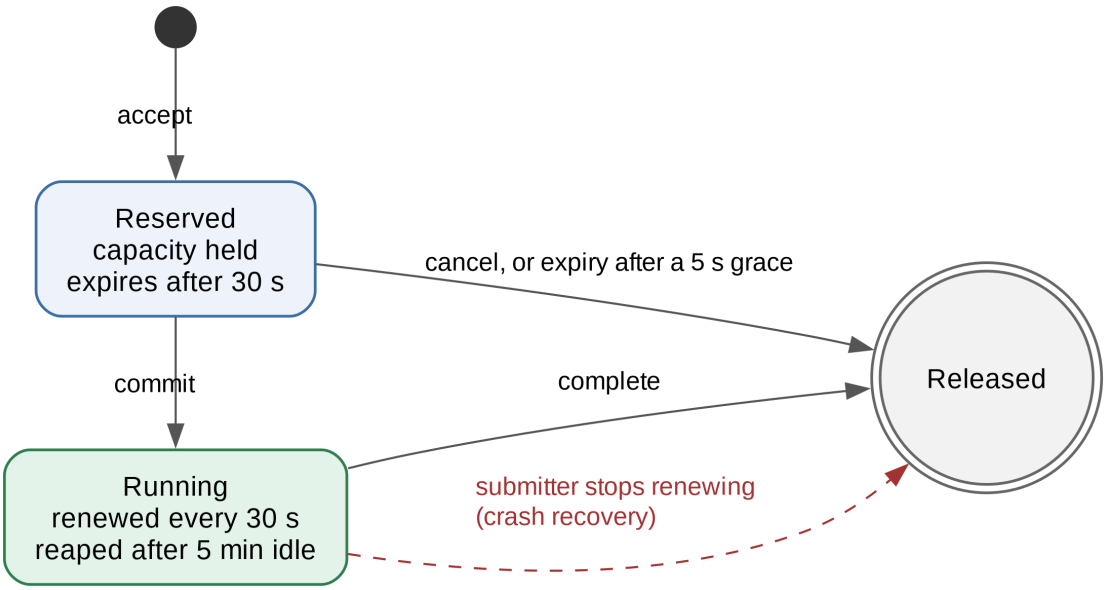

Figure 4.3: Lease states. Recovery is a consequence of expiry rather than a separate mechanism.

An `accept` reserves capacity for 30 seconds with a 5 second grace period; a `commit` promotes the reservation to running, renewed every 30 seconds and reaped after 5 minutes without renewal; `complete` releases it. A sweeper runs every 2 seconds.

Two details carry the correctness argument. First, the admission check and the lease insertion happen **under a single lock**, so two submitters racing for the last slot cannot both succeed. Second, the concurrency limit counts reserved and running leases together, deliberately, so a burst of submitters cannot slip through the window between accept and commit.

Recovery then needs no additional machinery. A submitter that dies stops renewing and the sweeper reclaims its slot. A worker that dies stops answering health checks, drops out of the healthy set after 45 seconds, and its in-flight connection fails, which the coordinator treats as a failed attempt and reschedules elsewhere.

4.4 Reproducing the Environment

Import scanning. The client extracts imports with a line-oriented regular expression rather than a full parse. This is worth stating precisely, because the consequence is a real limitation: only the top-level module token of each import is seen. Each name is then classified as a local file, a standard library module, or an external dependency, and external names are resolved to package names through a mapping table that handles cases such as `sklearn` to `scikit-learn` and `PIL` to `Pillow`.

The limitation, stated plainly. A third-party import absent from that table resolves

to nothing and is silently omitted from the closure. There is no package index lookup. A `uv.lock` file, which replaces detection entirely when present, and an explicit `--pkg` flag are the escape hatches.

The keystone. The generated flake pins an *exact* nixpkgs revision, not a branch. The reasoning is the foundation of the entire caching layer. A branch resolves at build time, so two nodes building the same script on different days would produce different store paths; because the cache is keyed on the store path, the system would then re-ship a multi-hundred-megabyte archive for every job. Pinning an exact revision makes the store path a stable, content-derived name for an environment, identical across every machine and across time. Section 5.3 measures what that buys.

Two deliberate exceptions exist, both restricted to x86_64 Linux: scikit-learn is pinned to a specific wheel because the package set lags the version targeted, and torch is fetched as a CUDA wheel set inside a *fixed-output derivation* with a recorded hash, because building it from source takes hours.

4.5 Closure Transfer and Cache Reuse

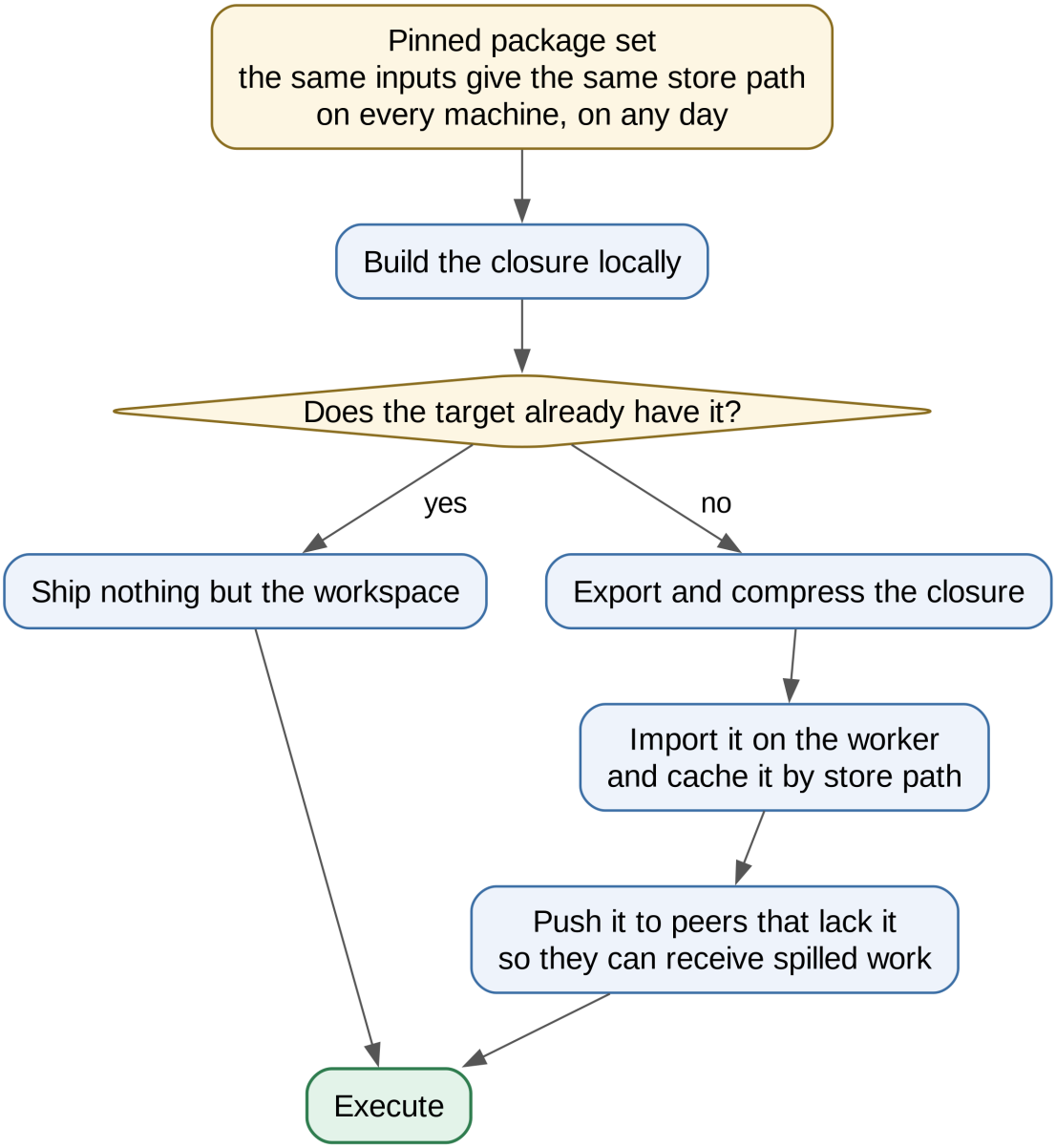

Figure 4.4: The store path is the cluster-wide cache key. Most submissions ship no environment at all.

Before uploading, the client asks the target whether it already holds the closure. Only on a negative answer is the closure exported and compressed.

There are in fact **two different questions** a node can be asked, and both are needed. The first is whether the archive is in its cache. The second is whether the closure is *materialised*, that is, whether its entry point exists on disk. A deployment with a shared store volume has the closure present without ever having cached an archive, so re-sending to it is wasted work; on a per-node store the entry point appears only after a real import.

Either predicate alone is wrong for one of the two layouts.

After an upload, the worker pushes the closure to peers that lack it. This is not merely an optimisation: work is only ever spilled to peers that already hold the closure, so the broadcast is what makes the cluster usable for the distribution described in Section 4.7.

4.6 Execution and Isolation

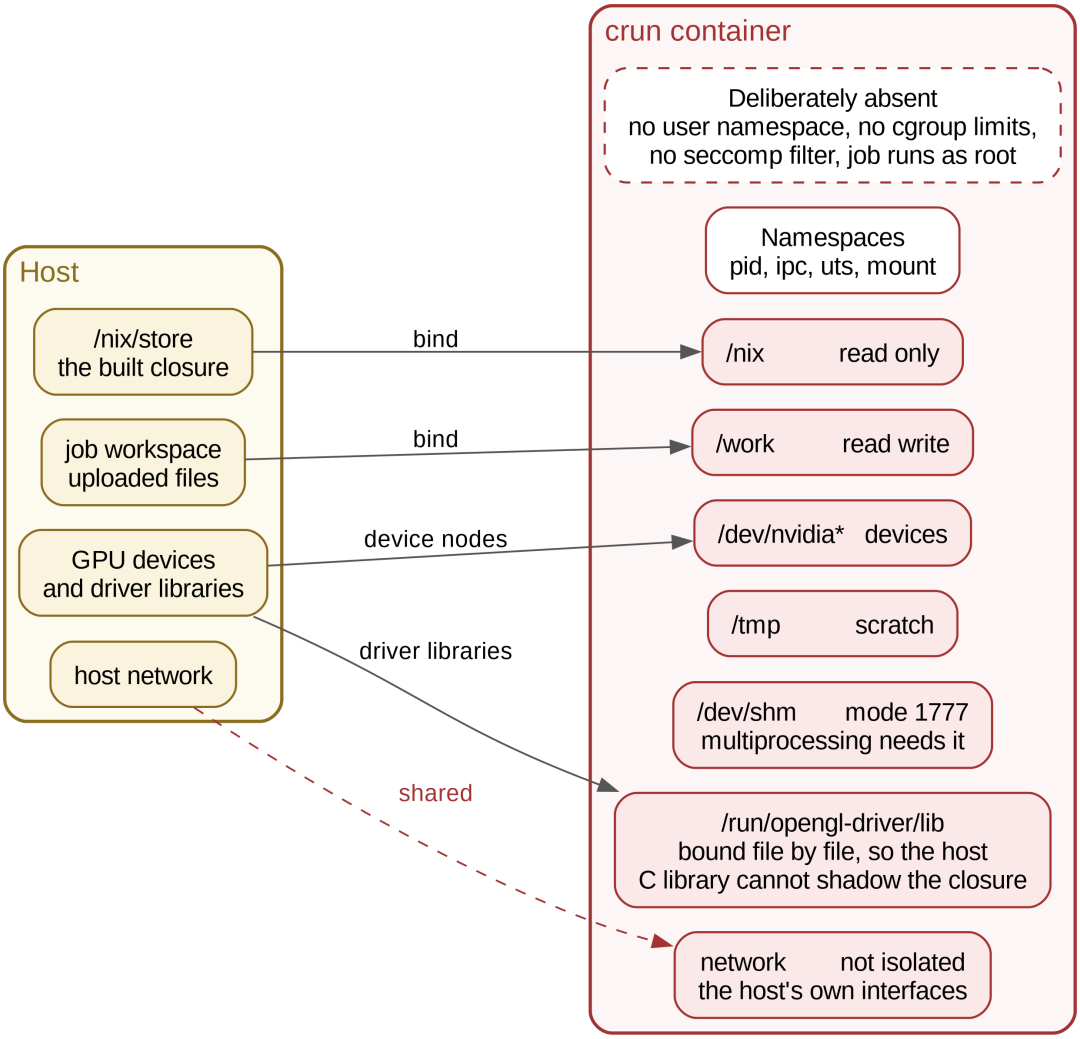

Figure 4.5: What the job can and cannot see. The absences are as deliberate as the mounts.

Jobs run under `crun` in an OCI bundle: the store read-only, the workspace read-write, and temporary filesystems for scratch space. Shared memory is mounted world-writable specifically because Python's `multiprocessing` maps POSIX shared memory for its semaphores and fails without it.

GPU support mounts the device nodes and then binds the driver libraries *file by file* rather

than binding the host library directory, because binding the directory would place the host C library ahead of the closure's on the search path. Without this, torch silently falls back to the CPU, which is indistinguishable from a working run except by its speed.

What the sandbox does not do. There are no cgroup limits, no seccomp filter, no capability set, no user namespace and no network namespace. The job runs as root with access to the host network. The namespaces in use are process, IPC, hostname and mount. If `crun` is absent the daemon warns and runs the job unisolated: the sandbox is a hardening layer, not a correctness requirement.

Results are diffed rather than copied wholesale. Only files created or modified during the run are returned, with deletions carried in a manifest, so a job cannot overwrite the submitter's own sources. Both the extraction on the worker and the extraction on the submitter refuse any path that escapes the target directory.

4.7 Transparent Interception

This is the system's distinguishing feature.

Delivery. The shim is a Python file embedded in the binary. It is appended to the workspace archive under a hidden directory using an archive transform, so the user's project on disk is never modified, and the worker points the interpreter's module path at that directory. CPython imports it automatically before the user's first line. Nothing is installed on any node.

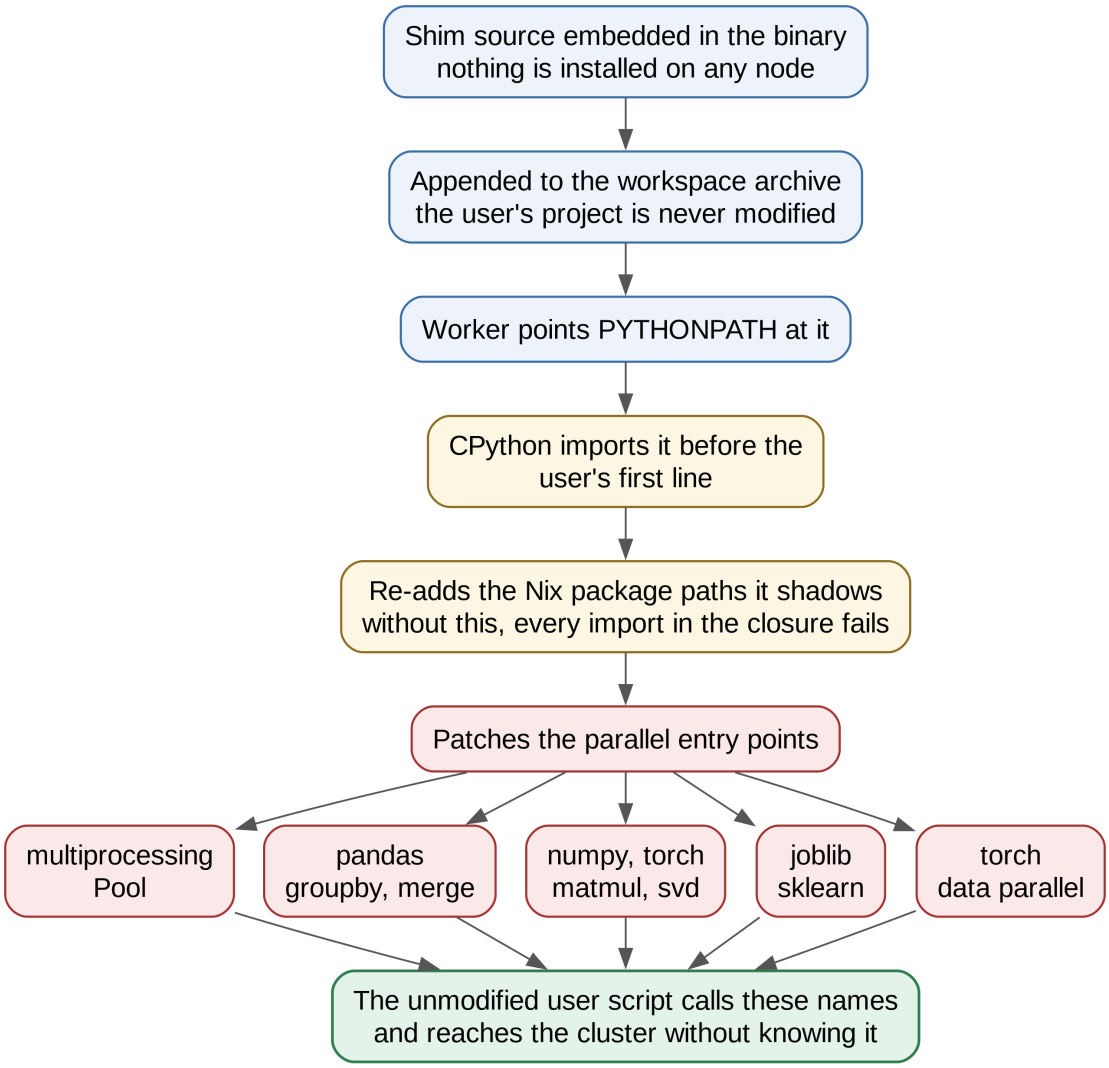

Figure 4.6: How the shim reaches the interpreter and what it replaces.

A hazard worth recording. Nix ships its own module of the same name, and only the first on the path is imported. The shim would therefore suppress the one that adds the environment's package directories, and every import in the closure would fail. The shim replays that work explicitly before doing anything else.

Patching is deferred. Each library is patched on its first import rather than when the shim installs itself. This matters: importing torch costs roughly 1.7 seconds, and paying it up front charged that cost to every job whether or not it used torch. Section 5.4 measures the difference.

Table 4.1: Intercepted primitives and their distribution strategies

Primitive	**Strategy**
`multiprocessing.Pool`, `ProcessPoolExecutor`	Measure a few items locally, then race the local and remote halves
pandas `groupby().agg()`, `merge`, `join`	Hash shuffle on the key
pandas `read_csv`, `read_parquet`	Chunked out-of-core reads cut at record boundaries
`numpy.matmul`, `dot`, `tensordot`	Split into row blocks, concatenate at the origin
`numpy.linalg.svd`, `eig`	Whole matrix offloaded to one worker
`joblib.Parallel`, scikit-learn `n_jobs`	One batch per request
torch training loop	Gradients averaged across ranks

Why the shuffle is exact. Rows are bucketed by a hash of the grouping or join key, so equal keys always fall in the same bucket and therefore land on the same node. Every key is complete on exactly one node, so each node's aggregation is a *complete* aggregation and the origin combines the partials with a plain concatenation. There are no combiners, and consequently *any* aggregation specification works, including those that have no combiner form at all. Bucket zero is kept local.

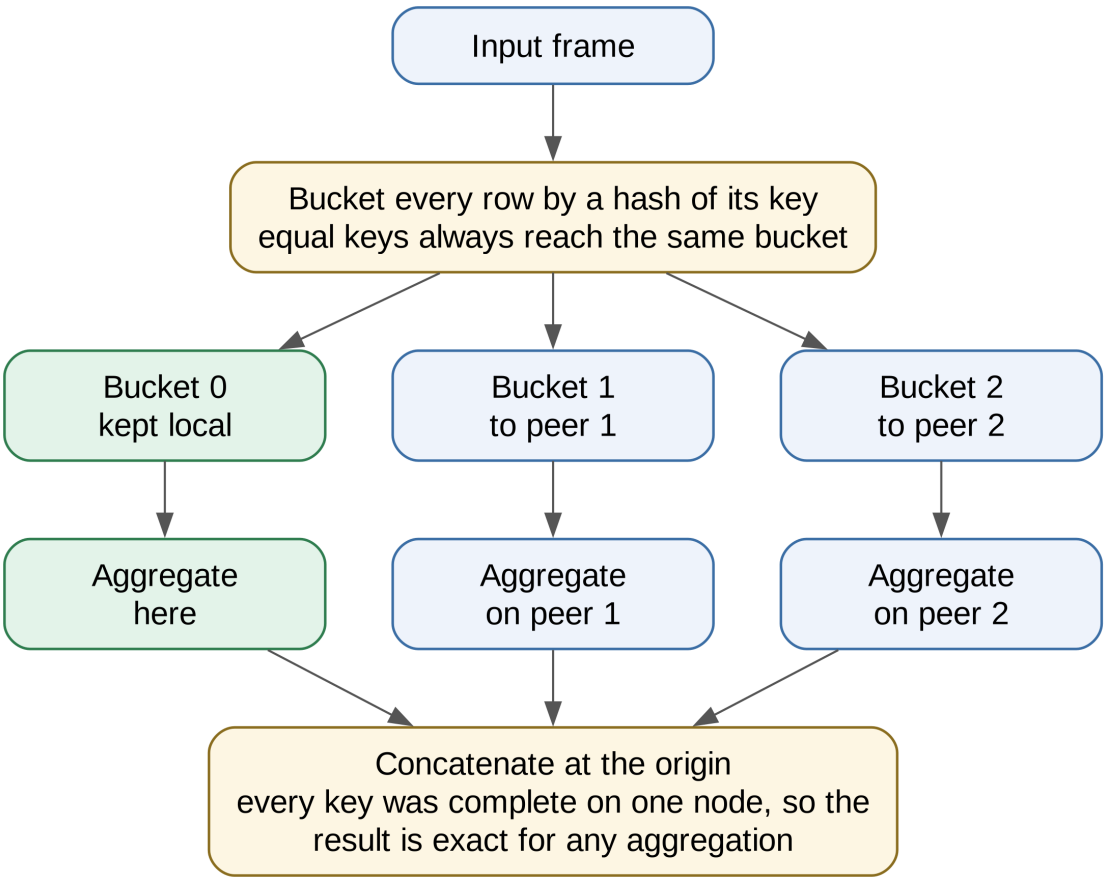

Figure 4.7: Hash shuffle. Correctness follows from key locality, not from combiner algebra.

Data-parallel training without a socket mesh. Rather than let torch build its rank-to-rank mesh, which requires direct connections on its own ports between every pair of ranks, each rank posts its gradient to the lead node's ordinary daemon port and receives the full set once every rank has arrived. The daemon acts as a blackboard: it never interprets the payload, and averaging happens in the shim. This trades a little bandwidth for working on any network where the daemon port already works, which is the only assumption the rest of the system makes.

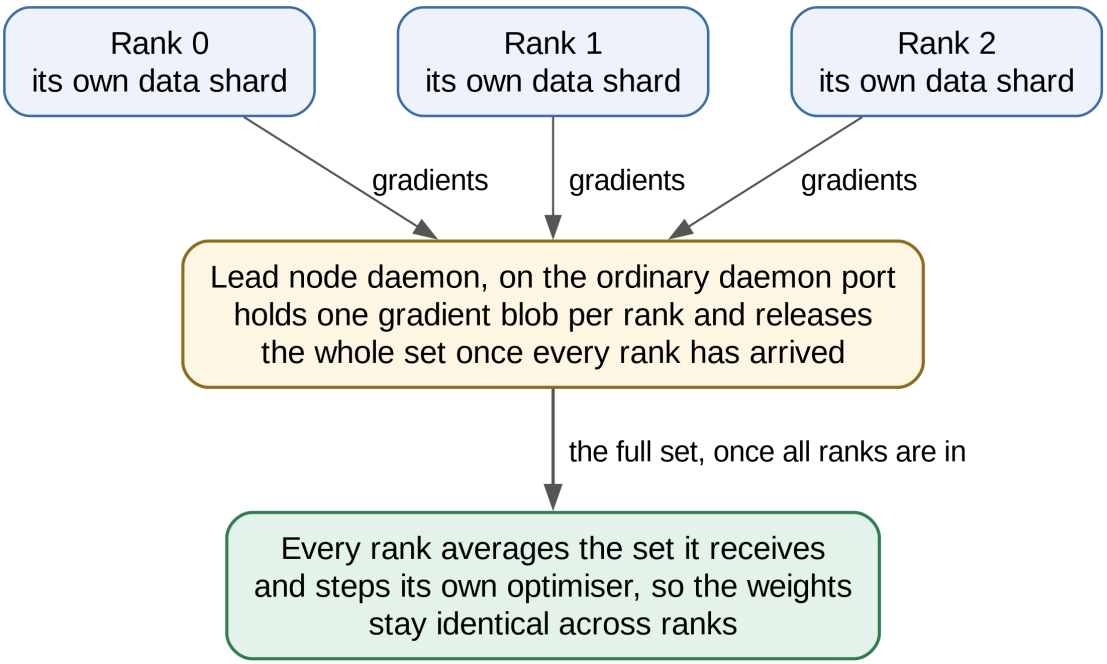

Figure 4.8: Gradient exchange over the daemon port.

4.8 The Cost Model

Interception is only worthwhile when the cluster actually wins, so every intercepted call is gated. Writing b for the payload in bytes, ϕ for its arithmetic intensity in floating-point operations per byte, B for measured link bandwidth and K for the number of participating nodes, work is distributed only when

$$\underbrace{\frac{b\phi}{R}}_{\text{compute locally}} > \underbrace{\frac{b}{B}}_{\text{move it}} + \underbrace{\frac{b\phi}{RK} \times 1.3}_{\text{compute remotely}} \tag{4.2}$$

where R is the effective local throughput and the factor 1.3 charges 30% overhead against the ideal parallel speedup. Three guards apply before the arithmetic: at least two nodes, a payload of at least 32 MB, and a carve-out that refuses work of low arithmetic intensity below 512 MB.

Bandwidth is *measured*, not assumed: 64 MB of data is pushed through the real dispatch path and the result cached for five minutes. If the probe fails, the answer is to stay local.

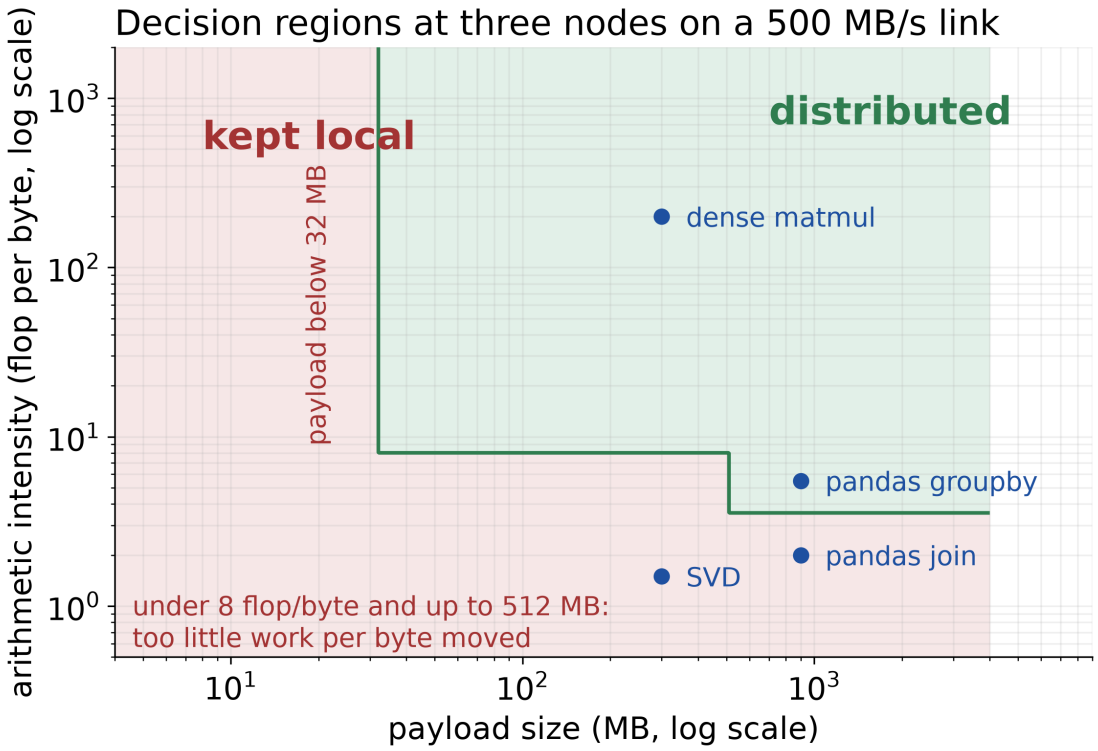

Figure 4.9: Decision regions from Equation 4.2. Dense matrix multiplication sits in the distributed region on paper but is held local by a separately calibrated model, because local BLAS beats the transport.

A second model calibrated against real BLAS throughput governs the NumPy path, and it produces a result worth highlighting because it is counter-intuitive: **dense matrix multiplication is never shipped**. Local BLAS runs at roughly 200 GFLOP/s, which no cluster transport can beat at practical sizes. The project treats this as a property to be tested rather than a missing feature: the demonstration suite fails if a matrix multiplication is ever seen leaving the node. Singular value decomposition, two orders of magnitude slower locally, does ship.

4.9 Fault Tolerance

The invariant is that using the cluster is never worse than not using it. Five independent layers enforce it.

1. **The race.** Chunks are dealt alternately to the local side and the cluster, and results fill a slot table first-wins. Idle local cores re-run any chunk the cluster has not yet returned, and the call returns only once the local side has covered every item. A completely dead cluster therefore degrades to a plain local pool.

2. **Per-primitive fallback.** Every intercepted call is wrapped; any exception falls back to the original implementation.

3. **Peer fallback.** A failed peer is skipped for the remainder of that chunk and the next candidate tried; if all fail, the work runs locally.
4. **Worker respawn.** A dead warm worker is restarted, falling back to a one-shot process if that fails.
5. **Rescheduling.** A failed job is placed again with backoff doubling from 2 seconds to a 2 minute ceiling. Tasks are never dropped.

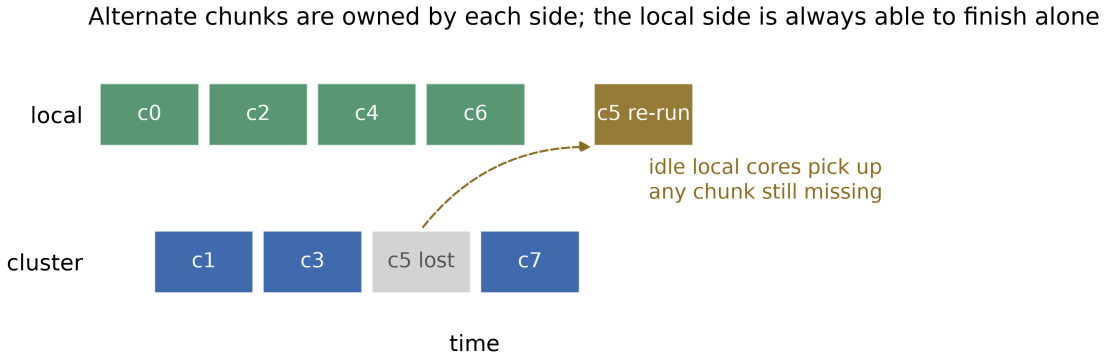

Figure 4.10: Racing local and remote work. The local side is always able to finish alone, which is what makes the guarantee unconditional.

Memory is bounded at the same time: a request larger than a node can hold is forwarded once to a healthier peer, and one exceeding 40% of available memory is split into sequential chunks rather than refused.

4.10 Resource Estimation

Four tiers are consulted, the first that yields a value winning: an explicit override; the largest peak memory from the last five runs of the same script, plus 20%; the sum of input file sizes times three plus a per-library baseline; or the baseline alone. The default when nothing is known is 512 MB.

The historical tier closes a feedback loop. The worker samples the job's process tree while it runs and records the peak, and that measurement sizes the next run's admission request.

4.11 Test Workloads

Ten dispatch scripts exercise placement and concurrency. Five demonstration workloads cover scikit-learn with `joblib`, NumPy linear algebra, out-of-core pandas, torch data-parallel training and file synchronisation. A three-node container lab and a four-container rig with a shared store provide the execution environments used in Chapter 5.

Chapter 5

Results and Discussion

5.1 Test Bed

All measurements were taken on 2026-08-25 on a single host running Linux 7.1.8 with 20 logical cores and 16 GiB of free memory, with three worker daemons in containers on ports 38081–38083 and the submitting daemon on port 38080.

One caveat governs this chapter. All three workers are containers on one machine and therefore share one CPU, one memory system and the loopback interface. This test bed measures correctness, overhead, cache behaviour and fault tolerance honestly. It *cannot* measure multi-machine speedup, because moving processor-bound work between containers on one host cannot make that host faster. **No speedup figure is claimed anywhere in this chapter.** Section 5.10 lists what that leaves out. The same harness runs unchanged against separate machines.

5.2 Functional Verification

The full suite was run with every optional Python library present.

Table 5.1: Test suite results

Outcome	Count	Notes
Passed	246	across 19 packages
Failed	0	
Skipped	3	two need an integration flag, one needs an NVIDIA GPU

Five interception tests had been skipping silently on any machine without pandas, torch and scikit-learn installed, which includes the project's own continuous integration job. The paths they cover were therefore unverified in automation. With the libraries present they run and pass; Table 5.2 records what each establishes.

Table 5.2: What the interception tests establish

Test	**Property established**
Hash shuffle matches local	`groupby` and all four join types are frame-equal to local pandas
NumPy interception	matmul splits into row blocks, SVD ships whole, results match
Cost model decisions	the decision boundaries hold at pinned bandwidths
Joblib backend distributes	an unmodified `RandomForestClassifier` reaches the cluster
Out-of-core integrity	chunk boundaries fall on record boundaries; memory stays bounded
Gradient synchronisation	two live ranks; each rank's gradient is exactly the mean
Race with dead remote	every remote chunk failing still yields correct results

5.3 Environment Cache

This is the strongest measured result and the direct payoff of pinning the package set so that a store path names an environment identically everywhere (Section 4.4). The same job was submitted five times.

Table 5.3: End-to-end time for repeated submissions of one environment

Submission	**Seconds**	**What was shipped**
First, closure absent on the worker	12.00	closure and workspace
Second to fifth, median	0.66	workspace only

An **eighteenfold reduction**. The second submission finds the closure already materialised and ships nothing but the project files.

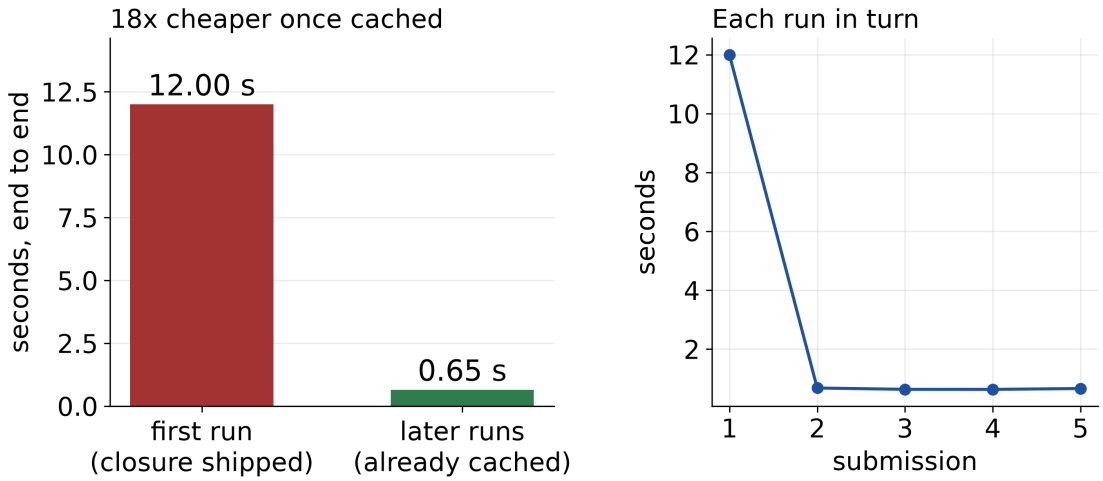

Figure 5.1: Cold and warm submission of the same environment.

5.4 The Cost of Leaving Interception On

Interception is on by default, so its cost when it decides *not* to distribute is the number that matters for everyday use.

Table 5.4: Whole-job cost of interception on a workload the cost model declines

Configuration	**Median seconds**	
Interception disabled	0.85	
Interception enabled	0.93	+9%

A separate measurement covers interpreter startup for a job that never imports a heavy library. The shim previously imported torch while installing itself, roughly 1.7 seconds, and charged it to every job. The project sets itself a budget of 3× for this overhead and was exceeding it sevenfold; the check that detects it is not run by continuous integration, so it had gone unnoticed. Deferring each library's patching to its first import corrects it.

Table 5.5: Interpreter startup overhead for a job that imports nothing heavy

Configuration	**Slowdown**	**Within the 3× budget?**
Eager patching (previous behaviour)	21.60×	no
Deferred patching (current)	1.20×	yes

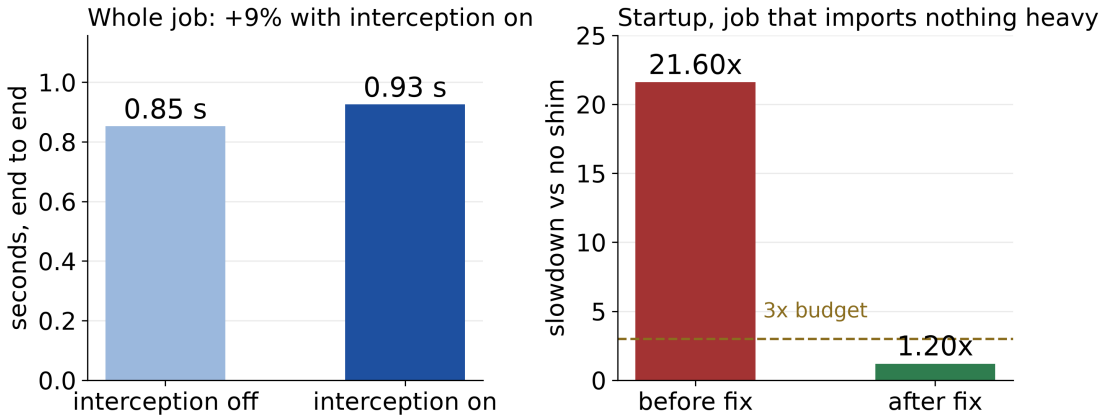

Figure 5.2: Left: cost over a whole job. Right: interpreter startup, before and after deferring library patching.

5.5 Work Crosses Node Boundaries

With distribution forced, all three workers logged incoming chunk requests (three, three and four respectively), confirming that interception, chunking and dispatch operate end to end across machines. No speedup is inferred from this, for the reason given in the test bed note.

5.6 Fault Tolerance

A twenty-item parallel map was fanned across the cluster and a worker container was killed while the computation was in flight.

Listing 5.1: Fault injection: a worker is killed mid-computation

```
killing worker on :38082 mid-flight
POOL-OK sum=2470
PASS: pool.map completed correctly despite a dead worker
```

The value 2470 is the correct sum of squares of the integers 0 to 19. The run completed with correct results despite losing a node during the computation, demonstrating the guarantee of Section 4.9 end to end.

5.7 Sandbox Cost

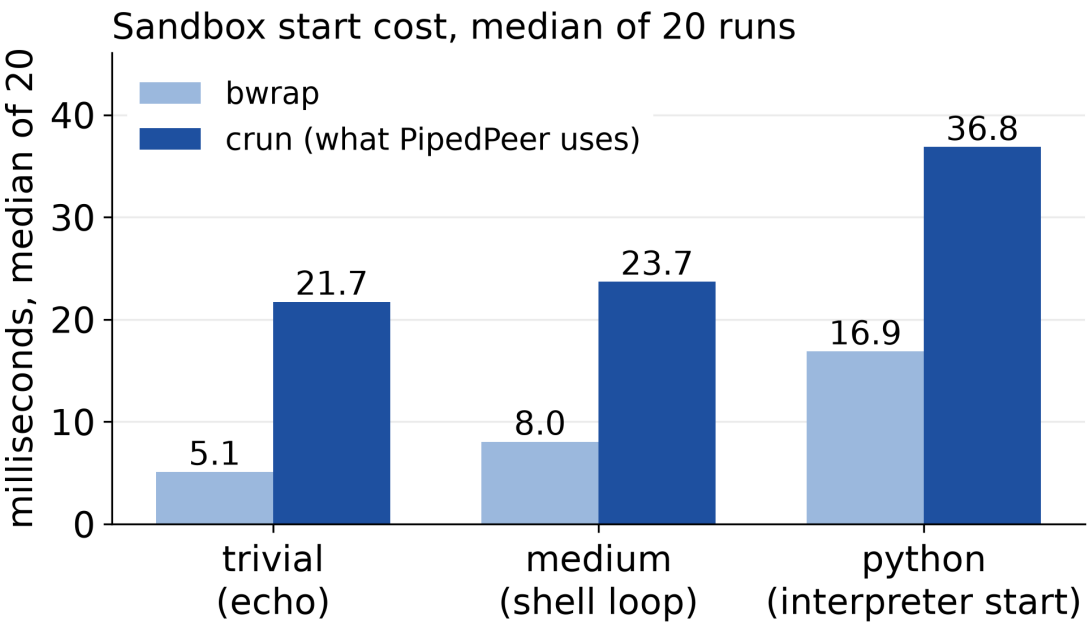

Figure 5.3: Sandbox start cost, median of 20 iterations.

The OCI runtime costs roughly 20 ms more per job than the lighter alternative. Measured against a 0.66 second warm job, let alone a 12 second cold one, this is not a figure worth optimising, and the OCI device model is what makes GPU passthrough possible.

These are the first valid numbers this benchmark has produced. It previously gave neither sandbox a root filesystem, so no shell existed inside either and all sixty measured runs exited with an error; the published timings were the cost of failing to start rather than the cost of starting. It now refuses to write a report if any measured run fails.

5.8 Scale of the Implementation

Table 5.6: Implementation size

Measure	**Value**
Go source	24,700 lines across 73 files
Internal packages	19
HTTP endpoints	18
Embedded Python shim	approx. 1,900 lines
Test files	33
Passing test cases	246

5.9 Comparative Analysis

Table 5.7: PipedPeer against existing systems

Property	**Piped-Peer**	**Spark**	**Ray**	**Dask**	**BOINC**	**ssh**
Runs unmodified Python	yes	no	partial	partial	no	yes
No cluster to configure	yes	no	no	no	no	partial
No head or master node	yes	no	no	no	no	yes
Reproduces the environment	yes	partial	partial	partial	partial	no
Sandboxes the job	yes	no	no	no	partial	no
Reschedules on node loss	yes	yes	yes	yes	yes	no

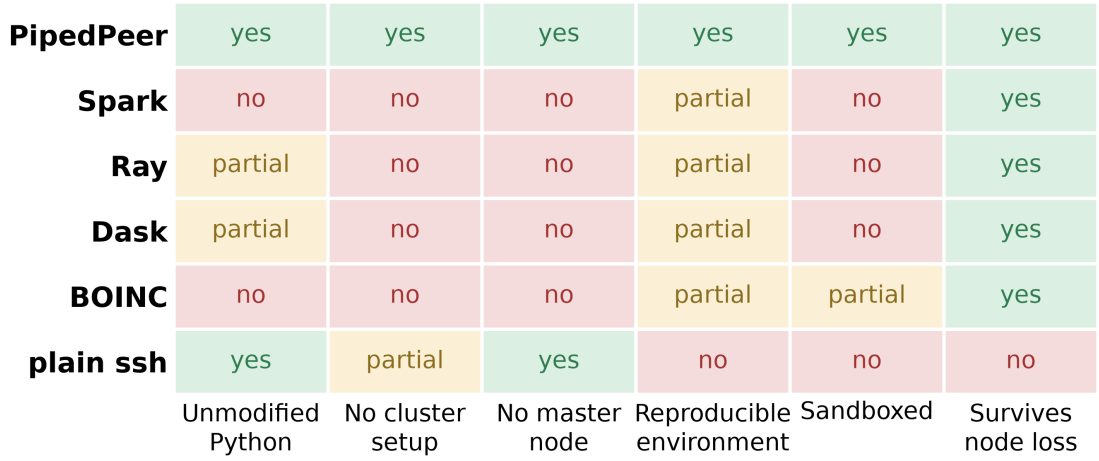

Figure 5.4: The same comparison in matrix form.

The comparison should be read carefully. Spark, Ray and Dask are more capable at scale, with mature schedulers, sophisticated fault tolerance and years of production use behind them; the axis on which PipedPeer differs is setup cost and code change, not raw capability. BOINC is closest in spirit, having pioneered volunteer computing, but it is a platform for specific projects each with its own server infrastructure rather than a general executor for arbitrary scripts.

5.10 What Was Not Measured

Recorded explicitly rather than left as gaps.

Table 5.8: Measurements not attempted, and why

Measurement	**Why it is absent**
Multi-machine speedup for parallel map and hash shuffle	every worker shares this one host's processor; needs separate machines
Scaling of data-parallel training across ranks	as above; correctness is covered by the gradient test
Peer discovery latency	the containers share the host's loopback, so the figure would not be meaningful
GPU placement and VRAM-aware scoring	no GPU on this host
The five demonstration workloads	they pull multi-gigabyte CUDA environments; not attempted here

Chapter 6

Conclusion and Future Work

6.1 Conclusion

PipedPeer runs an unmodified Python script on another machine with no cluster to configure, reproduces its environment exactly, and distributes the parallel work inside it without the user touching the source. The contributions are:

1. **The store path as a cluster-wide cache key.** Pinning an exact package set makes an environment's content-addressed name identical on every machine and on any day, which turns environment transfer from a per-job cost into a once-per-cluster cost. Measured at 12.00 seconds cold against 0.66 seconds warm.
2. **Transparent interception governed by a measured cost model.** The parallel primitives a script already uses are substituted at import time and distributed only when moving the data costs less than computing it locally, with link bandwidth measured rather than assumed.
3. **Exactness without combiners.** Hash-shuffling on the key means every key is complete on one node, so any aggregation works exactly, including those with no combiner form.
4. **Placement without a scheduler.** Hard filters followed by a bounded score, executed in the submitting client, with a three-phase lease providing atomic reservation and crash recovery through expiry.
5. **An unconditional degradation guarantee.** Five independent layers ensure that a remote path always has a working local path beneath it; a worker killed mid-computation left the run to complete with correct results.

The part most worth defending is the cost model. A system that distributed everything it could intercept would be slower than plain Python on most real workloads, because the transport rarely beats a modern BLAS or a warm page cache. Measuring bandwidth, declining to distribute dense matrix multiplication, and treating that refusal as a property to be tested rather than a missing feature is what makes always-on interception defensible.

6.2 Limitations of the Current System

- **Local network scope.** Discovery is a UDP probe on the local subnet; anything wider requires the registry or manually configured peers.
- **Python only.** Other runtimes would need their own environment packaging and their own interception layer.
- **Linux workers.** The sandbox and the store are Linux-specific.
- **The submitter must survive the job.** It holds the lease and receives the output stream.
- **Silent dependency omission.** A third-party import absent from the resolver's table is dropped from the environment without a diagnostic (Section 4.4). This is the most user-visible sharp edge.
- **A thin sandbox.** No resource limits, no system call filtering and no user namespace (Section 4.6).
- **Evaluation on one host.** No speedup figure is claimed (Section 5.10).

Defects found while preparing this report

Three defects were found and corrected while the system was being documented, which is itself an argument for writing the measurements down:

1. The shim imported torch while installing itself, taxing every job by about 1.7 seconds and exceeding the project's own never-slower budget sevenfold. The tripwire that detects this is not run by continuous integration.
2. The sandbox benchmark gave neither sandbox a root filesystem, so all sixty measured runs failed and the published timings measured the cost of failing rather than the cost of starting.
3. The laboratory scripts were not idempotent and used a shared temporary directory as a job workspace, so the fault-tolerance demonstration could not be run a second time.

6.3 Security

PipedPeer assumes that every peer on the network is trusted. **There is no authentication, no authorisation and no transport encryption.** Any host able to reach the daemon port can execute arbitrary code as root on that machine, read the output and workspace of any job, and remove nodes from the directory. This is a deliberate scope decision for a

tool intended for a network the user controls, not an oversight, and it is documented as such in the repository. It is also the single largest barrier to running the system anywhere else, and it is why authentication heads the list below.

6.4 Future Work

- **Authentication and an encrypted transport.** A prerequisite for any deployment beyond a trusted local network.
- **Discovery beyond the subnet**, by distributed hash table or traversal of network address translation, removing the dependence on a registry.
- **Locality-aware placement.** Placement currently scores capacity only. Accounting for which node already holds the input data, as iDDS [19] does, would reduce transfer for repeated work.
- **Topology-aware placement.** Nodes are scored independently on scalar metrics, so the scorer is blind to network distance. A graph-structured representation [25] would address this.
- **Submitter fault tolerance**, allowing a submitter to disconnect and collect results later.
- **Additional runtimes** beyond Python.
- **Measurement on separate machines**, to obtain the speedup figures this evaluation deliberately does not claim.

Bibliography

[1] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Spark: Cluster computing with working sets," *Proc. 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*, 2010.

[2] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, and I. Stoica, "Ray: A distributed framework for emerging AI applications," *Proc. 13th USENIX OSDI*, 2018, pp. 561–577.

[3] D. P. Anderson, "BOINC: A system for public-resource computing and storage," *Proc. 5th IEEE/ACM Workshop on Grid Computing*, 2004, pp. 4–10.

[4] M. Rocklin, "Dask: Parallel computation with blocked algorithms and task scheduling," *Proc. 14th Python in Science Conference (SciPy)*, 2015, pp. 130–136.

[5] E. Dolstra, M. de Jonge, and E. Visser, "Nix: A safe and policy-free system for software deployment," *Proc. 18th LISA*, 2004, pp. 79–92.

[6] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," *Proc. 6th USENIX OSDI*, 2004, pp. 137–150.

[7] P. Maymounkov and D. Mazières, "Kademlia: A peer-to-peer information system based on the XOR metric," *Proc. 1st IPTPS*, 2002, pp. 53–65.

[8] Open Container Initiative, "OCI Runtime Specification, v1.0.2," 2021. [Online]. Available: https://github.com/opencontainers/runtime-spec.

[9] Synadia Communications, "NATS: Cloud native messaging." [Online]. Available: https://nats.io.

[10] S. Sekigawa, C. Sasaki, and A. Tagami, "Web application-based WebAssembly container platform for extreme edge computing," *Proc. IEEE GLOBECOM*, 2023, pp. 3609–3614.

[11] M. N. Hoque and K. A. Harras, "WebAssembly for edge computing: Potential and challenges," *IEEE Commun. Standards Mag.*, vol. 6, no. 4, pp. 68–73, Dec. 2022.

[12] S. Kakati and M. Brorsson, "A cross-architecture evaluation of WebAssembly in the cloud-edge continuum," *Proc. IEEE/ACM CCGrid*, 2024, pp. 337–346.

[13] Y. Li and S. Fujita, "Design of Elixir-based edge server for responsive IoT applications," *Proc. IEEE CANDARW*, 2023.

[14] Y. Zhou and X. Chen, "A synergistic Elixir-EDA-MQTT framework for advanced smart transportation systems," *Computers*, vol. 16, no. 3, 2024.

[15] J. Bachmeier, V. Yussupov, J. Henß, and H. Koziolek, "WebAssembly with wasi-nn for edge machine learning inference: Experiences and lessons learned," *Proc. ECSA*, 2025, pp. 343–359.

[16] S. E. Khelifa, M. Bagaa, A. O. Messaoud, and A. Ksentini, "Case study of WebAssembly runtimes for AI applications on the edge," 2024.

[17] A. Bergström, "Programming models for failure-transparent distributed systems," M.S. thesis, KTH Royal Institute of Technology, Stockholm, 2025. [Online]. Available: https://www.diva-portal.org/smash/get/diva2:2013047/FULLTEXT01.pdf.

[18] A. Meurer, S. Hoßfeld, R. Gommers, and the Consortium for Python Data API Standards, "Python array API standard: Toward array interoperability in the scientific Python ecosystem," *Proc. SciPy*, 2023. [Online]. Available: https://proceedings.scipy.org/articles/gerudo-f2bc6f59-001.

[19] W. Guan, T. Maeno, F. Barreiro Megino, T. Wenaus, and A. Klimentov, "iDDS: Intelligent distributed dispatch and scheduling for workflow orchestration," arXiv:2510.02930, 2025. [Online]. Available: https://arxiv.org/abs/2510.02930.

[20] Y. Deng, Z. Chen, X. Chen, and Y. Fang, "Task offloading in multi-hop relay-aided multi-access edge computing," *IEEE Trans. Veh. Technol.*, vol. 72, no. 1, pp. 1372–1376, Jan. 2023.

[21] H. He, X. Yang, X. Mi, H. Shen, and X. Liao, "Multi-agent deep reinforcement learning based dynamic task offloading in a device-to-device mobile-edge computing network," *Sensors*, vol. 24, no. 16, 2024.

[22] D. Yong, R. Liu, X. Jia, and Y. Gu, "Joint optimization of multi-user partial offloading strategy and resource allocation strategy in D2D-enabled MEC," *Sensors*, vol. 23, no. 5, 2023.

[23] L. Chen, Y. Wang, H. Zhang, and J. Liu, "Energy-efficient collaborative task offloading in multi-access edge computing based on deep reinforcement learning," *Ad Hoc Networks*, vol. 169, 2025.

[24] C. Li, Q. Chen, M. Chen, Z. Su, Y. Ding, D. Lan, and A. Taherkordi, "Blockchain enabled task offloading based on edge cooperation in the digital twin vehicular edge network," *J. Cloud Comput.*, vol. 12, no. 120, 2023.

[25] Z. Zhao, J. Perazzone, G. Verma, and S. Segarra, "Congestion-aware distributed task offloading in wireless multi-hop networks using graph neural networks," arXiv:2312.02471, 2023.